Python Fundamentals - Chapter 1

Introduction to Python

What is Python?

Python is a high-level, interpreted, general-purpose programming language. It is known for readable syntax and is used for automation, web applications, data analysis, machine learning, and many other tasks.

Why Python is popular

Python programs are often shorter and easier to read than equivalent programs in many other languages. Python has a large standard library and an active community that maintains useful third-party packages.

Interpreted execution

Python source code is commonly saved in files ending with .py. The Python interpreter executes the program. Many Python implementations compile source code to an intermediate form before executing it.

Common applications

Python is used for data cleaning, visualization, backend services, testing, scripting, scientific computing, and artificial intelligence. Libraries and frameworks extend what a Python program can do.

First program

The statement print("Hello, World!") displays the text Hello, World! in the output. Python uses indentation to group statements into blocks.

Key terms

A program is a sequence of instructions. A variable stores a reference to a value. A function is reusable code that performs a task. A module is a file containing Python definitions and statements.

Python Fundamentals - Chapter 2

Variables and Data Types

Variables

A variable name refers to an object. For example, age = 21 assigns the integer value 21 to age. Python does not require a separate variable type declaration.

Numbers

The int type represents integers, such as 10 or -4. The float type represents floating-point numbers, such as 3.14. Arithmetic operators include +, -, *, /, //, %, and **.

Strings

The str type represents text. A string can be written using single or double quotes. For example, language = "Python" creates a string. Strings support indexing, slicing, and methods such as lower() and upper().

Booleans

The bool type has two values: True and False. Boolean values are commonly produced by comparisons, such as 5 > 3, and used in conditional statements.

Type conversion

The functions int(), float(), and str() convert compatible values between types. For example, int("12") produces the integer 12. Invalid conversions can raise ValueError.

Checking types

The built-in function type() returns the type of an object. For example, type(3.5) is float. The function isinstance() can check whether an object belongs to a given type.

Python Fundamentals - Chapter 3

Operators and Conditional Statements

Arithmetic operators

Python supports addition (+), subtraction (-), multiplication (*), division (/), floor division (//), remainder (%), and exponentiation (**). Operator precedence determines the order of evaluation.

Comparison operators

The operators ==, !=, >, <, >=, and <= compare values. The result of a comparison is usually True or False.

Logical operators

The keywords and, or, and not combine or negate conditions. For example, age >= 18 and age < 60 checks whether age is between 18 and 59.

If statement

An if statement runs its indented block when its condition is true. Example: if marks >= 40: print("Pass"). The colon begins the block and indentation defines which statements belong to it.

Elif and else

An elif clause tests another condition if earlier conditions were false. An else clause runs when no earlier condition matched. These statements are useful for grading systems and menu choices.

Nested conditions

An if statement can appear inside another if statement. Nested conditions should be kept readable, and complicated conditions can often be simplified with logical operators.

Python Fundamentals - Chapter 4

Loops and Iteration

Why loops are useful

A loop repeats a block of code. Loops are useful when processing collections, performing repeated calculations, or applying the same operation to many values.

For loop

A for loop visits items from an iterable. Example: for number in [1, 2, 3]: print(number). This prints each number in the list.

Range function

The range() function generates a sequence of integers for iteration. For example, range(5) represents 0, 1, 2, 3, and 4. The ending value is excluded.

While loop

A while loop repeats as long as its condition remains true. For example, while count < 3: can repeat until count reaches 3. The loop body must update the relevant state to avoid unintended infinite loops.

Break and continue

The break statement exits the nearest enclosing loop. The continue statement skips the remaining statements in the current iteration and moves to the next iteration.

Loop else

Python loops may have an else block. The else block executes if the loop finishes normally without encountering a break statement.

Python Fundamentals - Chapter 5

Functions in Python

What is a function?

A function is a reusable block of code designed to perform a specific task. Functions reduce repetition, improve organization, and make programs easier to test and maintain.

Defining a function

The def keyword defines a function. Example: def greet(name): return "Hello, " + name. The function body is indented, and return sends a result back to the caller.

Calling a function

A function runs when called. For example, greet("Asha") returns the string Hello, Asha. The value "Asha" is an argument passed to the parameter name.

Parameters and arguments

Parameters are names in a function definition. Arguments are values provided when calling the function. Python supports positional arguments, keyword arguments, and default parameter values.

Return statement

The return statement ends function execution and provides a value to the caller. If a function finishes without returning a value explicitly, it returns None.

Why use functions?

Functions encourage code reuse, modular design, readability, and easier debugging. A well-designed function generally has a clear purpose and meaningful name.

Python Fundamentals - Chapter 6

Lists, Tuples, Sets, and Dictionaries

Lists

A list is an ordered, mutable collection. Example: fruits = ["apple", "banana", "mango"]. Items can be accessed by index, and methods such as append() and remove() modify the list.

Tuples

A tuple is an ordered, immutable collection. Example: point = (10, 20). Tuples are useful for grouping values that should not be reassigned through the tuple.

Sets

A set is an unordered collection of unique hashable elements. Example: unique_numbers = {1, 2, 3}. Sets are useful for removing duplicates and performing union or intersection operations.

Dictionaries

A dictionary stores key-value pairs. Example: student = {"name": "Asha", "marks": 88}. Values can be accessed with student["name"] or student.get("name").

List versus dictionary

A list is best for a sequence of items accessed by position. A dictionary is useful when each value has a meaningful key, such as a student name, course, or marks.

Iteration over collections

A for loop can iterate over lists, tuples, sets, and dictionaries. The items() method of a dictionary provides key-value pairs for iteration.

Python Fundamentals - Chapter 7

Files, Exceptions, and Modules

Reading text files

Python can open text files using open(). A common pattern is with open("notes.txt", "r", encoding="utf-8") as file: text = file.read(). The with statement closes the file automatically.

Writing text files

Use write mode "w" to create or overwrite a text file, or append mode "a" to add content. For example, file.write("Python notes") writes a string to an open file.

Exceptions

An exception is an event that interrupts normal execution. Common examples include ZeroDivisionError, ValueError, TypeError, and FileNotFoundError.

Try and except

A try block contains code that might fail. An except block handles a specified exception. For example, try converting input to int and catch ValueError if the input is not a valid integer.

Modules

A module is a Python file containing reusable functions, classes, or variables. The import statement makes a module available. Example: import math, followed by math.sqrt(16).

Packages and pip

A package groups related modules. The pip tool installs many third-party Python packages. Project dependencies are often listed in a requirements.txt file.

Python Fundamentals - Chapter 8

Python for Data Science and AI

NumPy

NumPy provides multidimensional arrays and numerical operations. It is commonly used for efficient numerical computation and as a foundation for other scientific Python libraries.

Pandas

Pandas provides DataFrame and Series data structures. It is used to load tabular data, handle missing values, filter records, and summarize datasets.

Matplotlib

Matplotlib creates plots such as line charts, bar charts, and scatter plots. Visualizations help explore trends, relationships, and distributions in data.

Scikit-learn

Scikit-learn offers tools for preprocessing, model training, and evaluation. It includes algorithms for classification, regression, clustering, and model selection.

Machine learning

Machine learning uses data to train models that make predictions or identify patterns. A typical workflow includes data preparation, train-test splitting, model training, evaluation, and improvement.

Retrieval-Augmented Generation

Retrieval-Augmented Generation, or RAG, combines document retrieval with a language model. A RAG application can extract text from a PDF, divide it into chunks, store searchable representations, retrieve relevant chunks, and generate an answer grounded in those chunks.

Testing a PDF chatbot

Useful questions for this document include: What is Python? What are Python functions? What is the difference between lists and dictionaries? How does a while loop work? What is RAG? A well-grounded chatbot should answer using the document content.